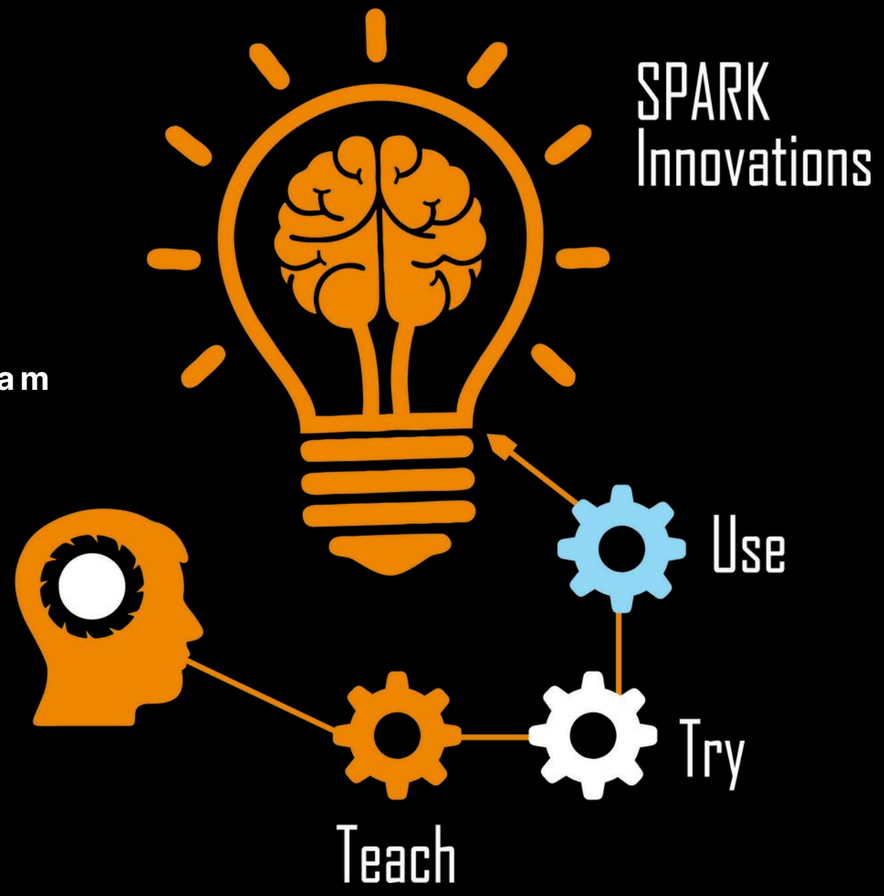
2026 SPARK ACADEMY

TRAIN FOR CHANGE, FROM SCIENCE TO PRACTICE

LIGHT WEIGHT MODEL DESIGN

Willem P.E. Boonzaier[1,2]
[1]Department of Medical Physics, University of the Free State, Bloemfontein, South Africa
[2]SPARK AI Training for African Medical Imaging Knowledge Translation Program

williebоonz@gmail.com



Introduction

Problem:

- **Many deep learning models require sophisticated hardware:**
  - **Dedicated memory (RAM)**
  - **GPU**
- **The larger the model the more the requirements**
- **Modern deep learning models generally have more than 20 million parameters.**
- **Take nnunet as example:**

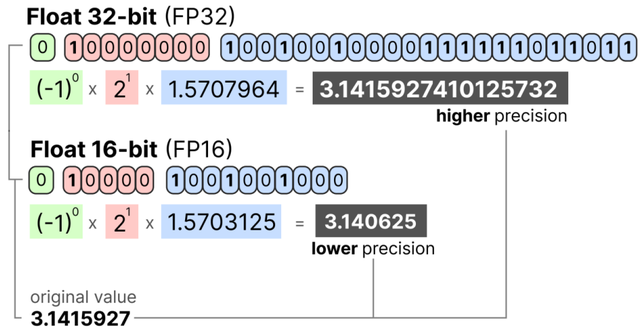
nnU-Net Parameter and Model Size Estimates

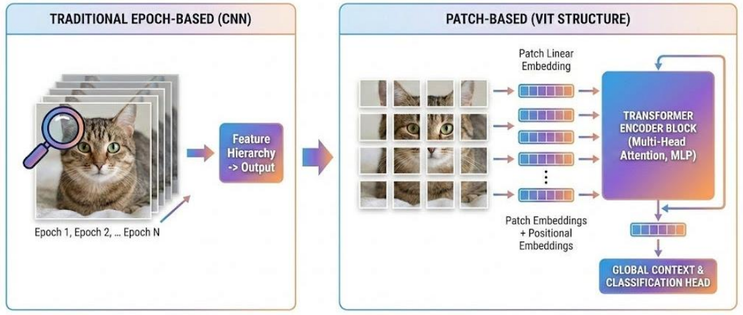
Configuration	Typical Parameters	Model Size (MB)
2D U-Net	~30M – 60M	~120MB – 240MB
2.5D U-Net	~35M – 70M	~140MB – 280MB
3D FullRes	~100M – 220M	~400MB – 880MB

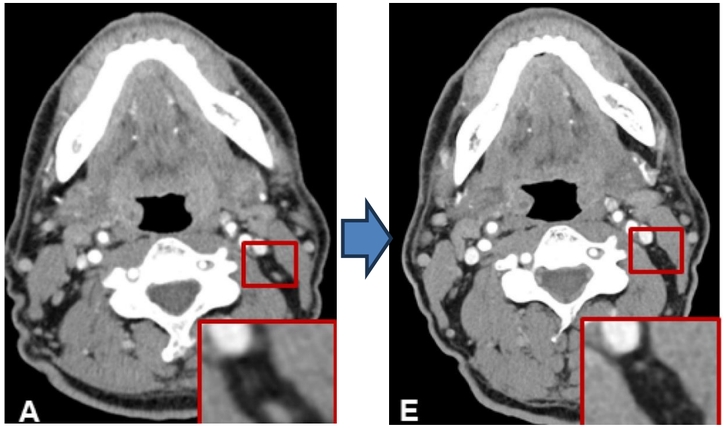
Introduction

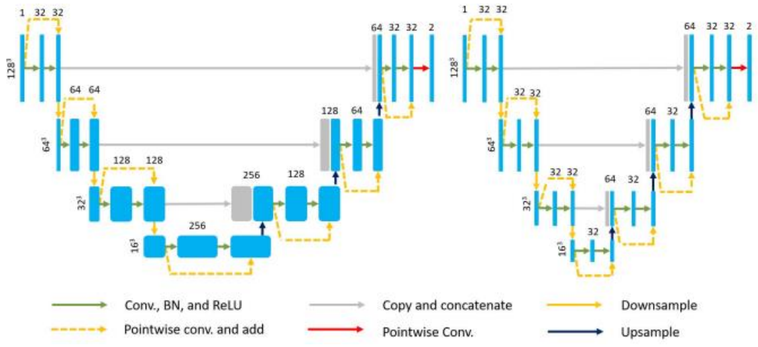
Solution:

- Light weigh models:
  - Reducing model parameter count
  - Using less memory
  - Improving speed
  - Adapting to CPU based environments





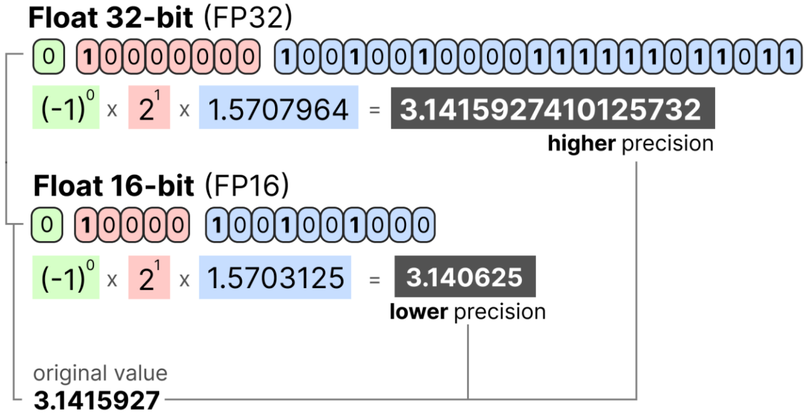




- Many Approaches exist:
  - Precision trimming:
    - Automatic mixed precision training
    - Post training quantization
  - Low resolution techniques:
    - Training at half resolution
    - 2D or 2.5D models instead of 3D models
  - Patch based techniques
  - Architecture adjustments
    - MIST-medical pocket net model
    - Filter width pruning

Precision trimming

- It is ideal to always have 32 bit float precision
  - If needed we have scope to use all the bits we have allocated to each parameter
- But what if we don't need it?
- Automatic Mixed Precision (AMP) is a technique that uses both 16-bit (FP16 or BF16) and 32-bit formats during training to maximize throughput without sacrificing model accuracy.



Benefits of AMP

Feature	FP32 Training	Mixed Precision (AMP)
Memory Usage	High (4 bytes per param)	Lower (~50% reduction in activations)
Speed	Standard	Up to 2-3x faster on Tensor Core GPUs
Batch Size	Limited	Can often double the batch size
Accuracy	Baseline	Matches FP32 (with proper scaling)

```python
scaler = torch.cuda.amp.GradScaler()

for data, target in dataset:
    optimizer.zero_grad()

    # Runs the forward pass in mixed precision
    with torch.cuda.amp.autocast():
        output = model(data)
        loss = loss_fn(output, target)

    # Scales loss, calls backward(), and unscales gradients
    scaler.scale(loss).backward()

    # optimizer.step() is wrapped to handle the scaled gradients
    scaler.step(optimizer)

    # Updates the scale factor for the next iteration
    scaler.update()
```

Precision trimming

- It is ideal to always have 32 bit float precision
  - If needed we have scope to use all the bits we have allocated to each parameter
- But what if we don't need it?
- Post training quantization

Why Use PTQ?

Feature	Standard (FP32)	Quantized (INT8)
Model Size	100 MB	~25 MB (4x reduction)
Latency	Standard	2x to 4x faster on many CPUs
Power Consumption	High	Significantly lower (critical for mobile)
Hardware Support	Universal	Excellent on ARM, Edge TPUs, and DSPs

Python

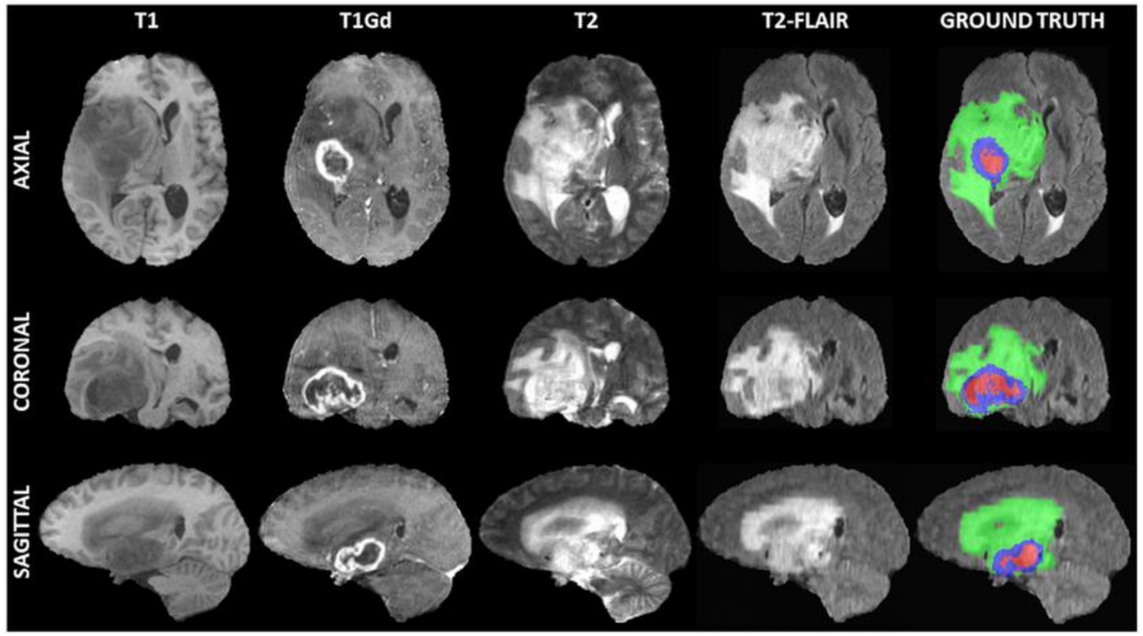
```python
import torch

# 1. Load model
model_fp32 = MyModel()
model_fp32.eval()

# 2. Define quantization configuration
model_fp32.qconfig = torch.ao.quantization.get_default_qconfig('fbgemm')

# 3. Fuse layers and prepare
model_prepared = torch.ao.quantization.prepare(model_fp32)

# 4. Calibrate with data
with torch.inference_mode():
    for x in calibration_dataloader:
        model_prepared(x)

# 5. Convert to INT8
model_int8 = torch.ao.quantization.convert(model_prepared)
```

GPU needs at least 16 bit float

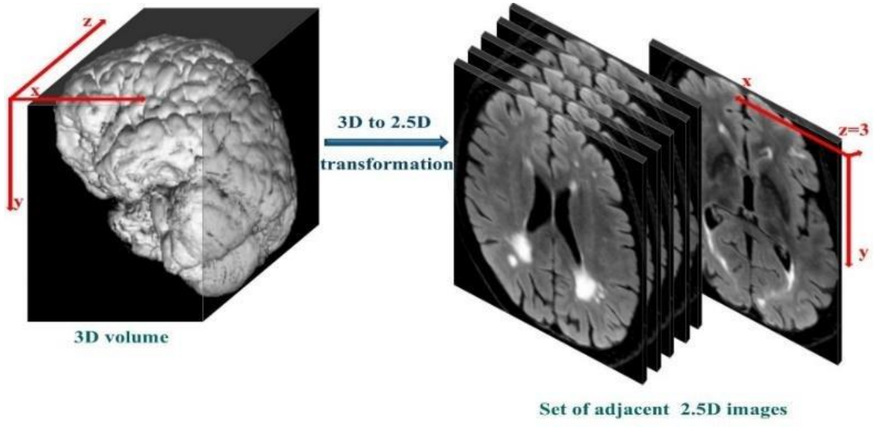
Low resolution training

- **The spatial resolution in many modern medical systems is better than what is required for AI accuracy.**

- **Not all slices have to do with one another:**
  - **For example the eye and the chin.**



Method	VRAM Cost	Context	Best For
Low Res	Depends on reduction factor	Global	Finding large structures/organs.
2D	Low	None (Z-axis)	Slice-by-slice classification.
2.5D	Medium	Local (Z-axis)	High-throughput segmentation.
Full 3D	Very High	Full Volumetric	Complex anatomy (vessels, nerves).

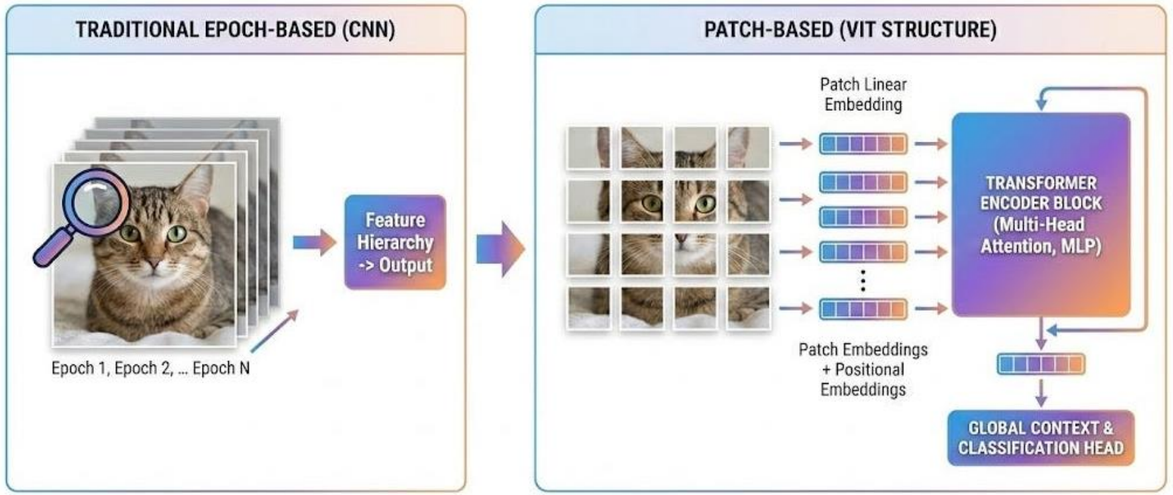


Patch based techniques

- **Dividing each image into smaller bits (patches)**

Implementation Benefits

Feature	Full-Volume Training	Patch-Based Training
GPU VRAM	Extremely High (often >48GB)	Low to Medium (can fit on 8GB-12GB)
Batch Size	Usually 1 (Stochastic)	Higher (4, 8, or 16), leading to stable gradients
Data Augmentation	Limited	High (each patch can be rotated/flipped independently)
Detail Preservation	Often requires downsampling	Maintains original voxel resolution

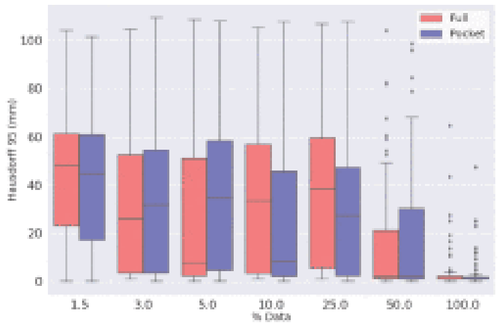


Python

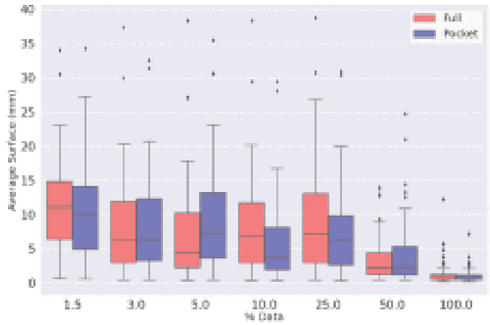
```python
from monai.transforms import (
    Compose, LoadImaged, EnsureChannelFirstd,
    RandCropByPosNegLabeld, ToTensord
)
from monai.data import Dataset, DataLoader

# Define the pipeline
transforms = Compose([
    LoadImaged(keys=["image", "label"]),
    EnsureChannelFirstd(keys=["image", "label"]),
    # Crops 4 patches of 96x96x96 per volume
    # 'pos=1, neg=1' ensures 50% of patches contain the label
    RandCropByPosNegLabeld(
        keys=["image", "label"],
        label_key="label",
        spatial_size=(96, 96, 96),
        pos=1, neg=1, num_samples=4
    ),
    ToTensord(keys=["image", "label"]),
])

# Load and Batch
data = [{"image": "img1.nii.gz", "label": "seg1.nii.gz"}]
ds = Dataset(data=data, transform=transforms)
loader = DataLoader(ds, batch_size=2) # Yields 8 patches per step
```

MIST MEDICAL POCKET MODEL

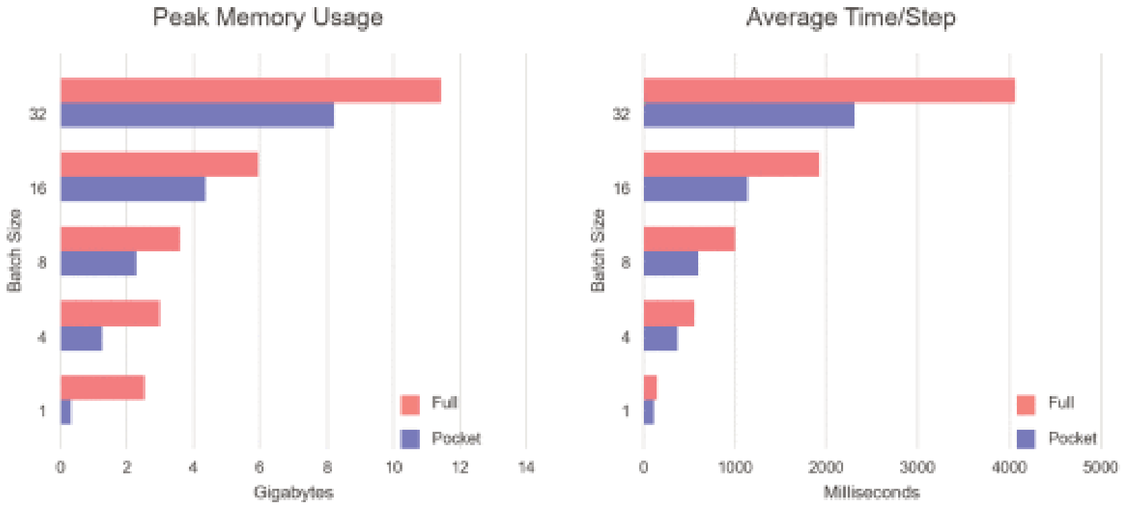
- **Architectural change limiting filters per conv layer**
- **https://github.com/mist-medical/MIST**

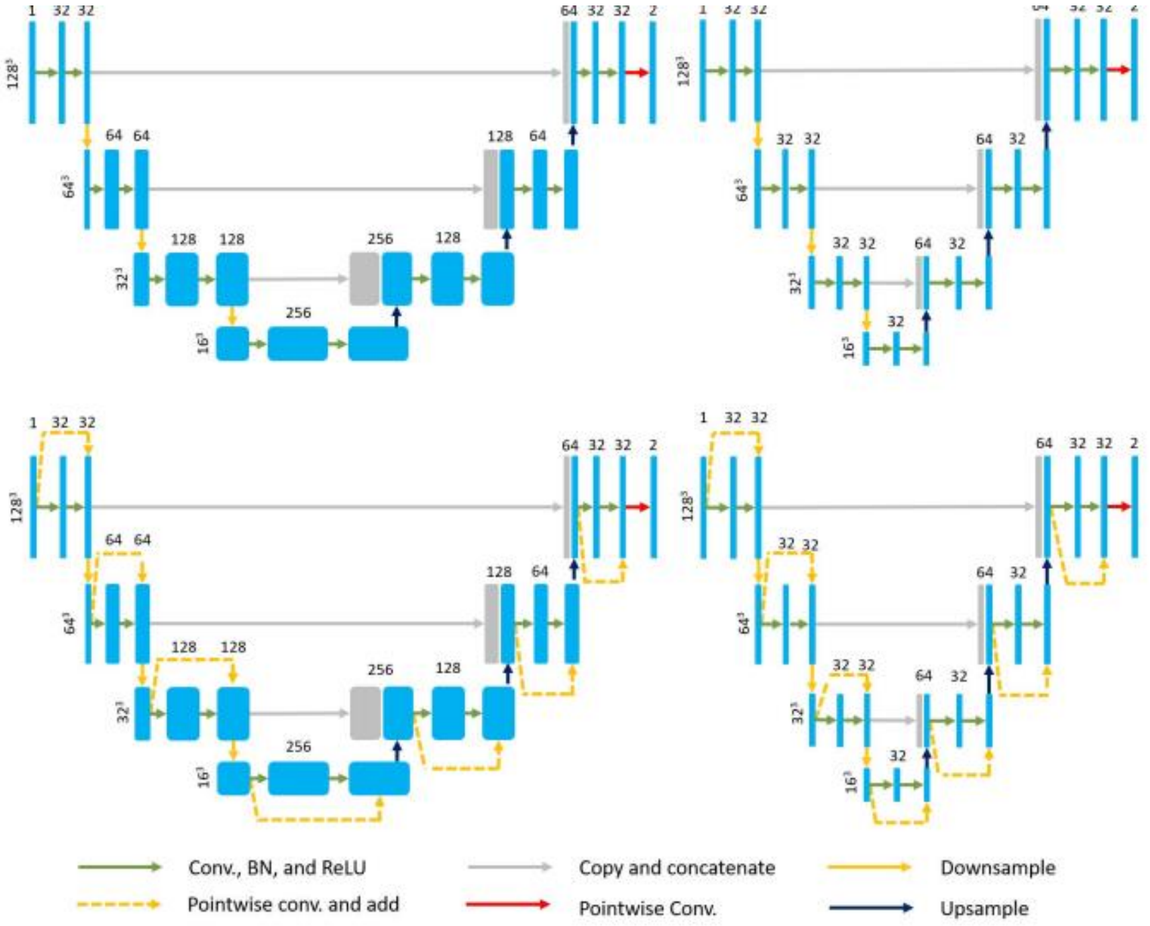


(c) Distributions of Hausdorff 95 distances on BraTS test set for a full U-Net and a pocket U-Net for varying training set sizes.



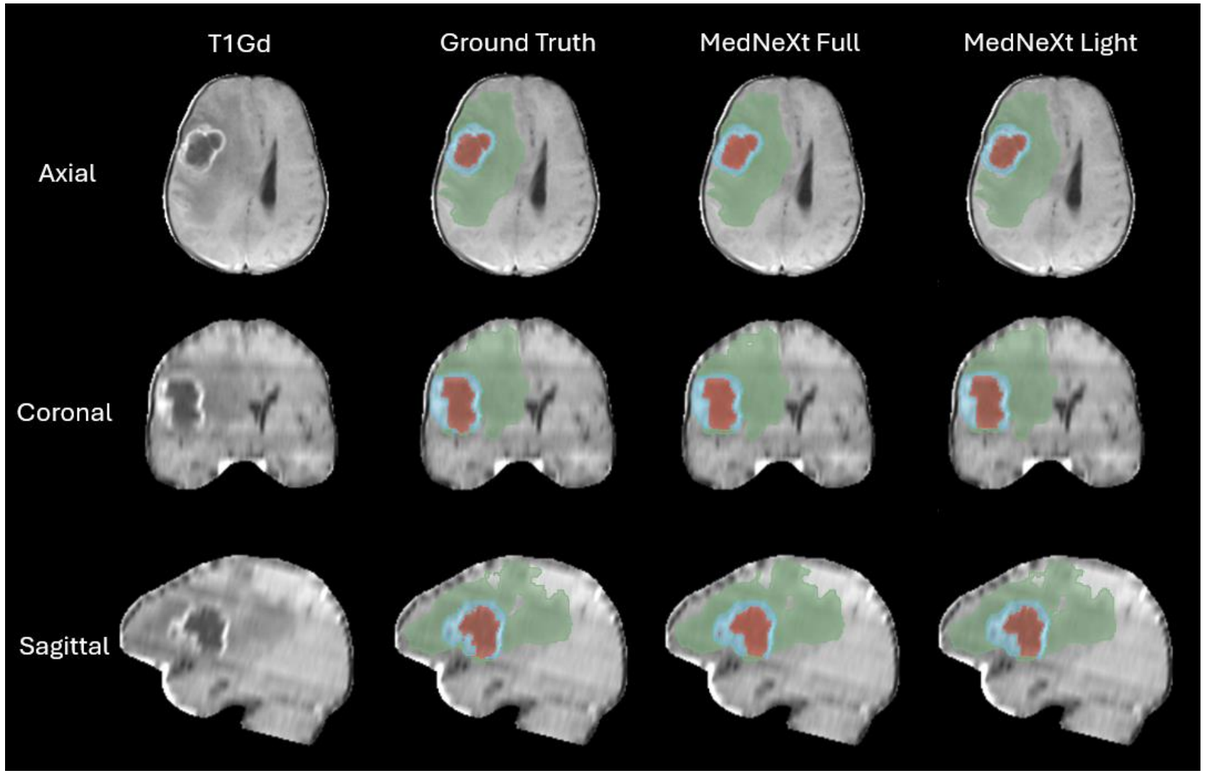
(d) Distributions of average surface distances on BraTS test set for a full U-Net and a pocket U-Net for varying training set sizes.

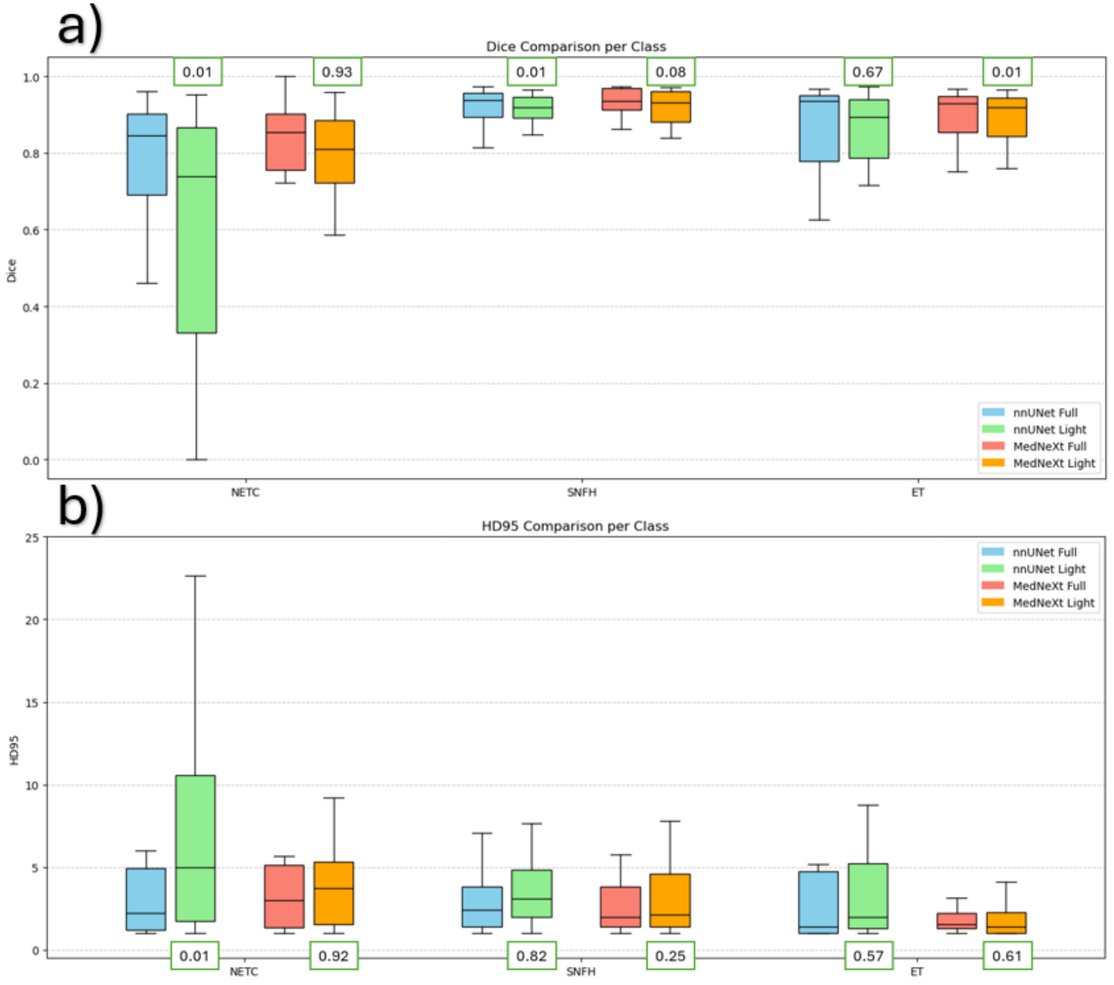




Model filter width pruning

- Architectural change limiting filters per conv layer
- Instead of having nnunet standard filter density of for translation of 32-64-128-256-512-512 I did 32-32-32-32-32-32
- 0.8GB to 0.004GB size reduction





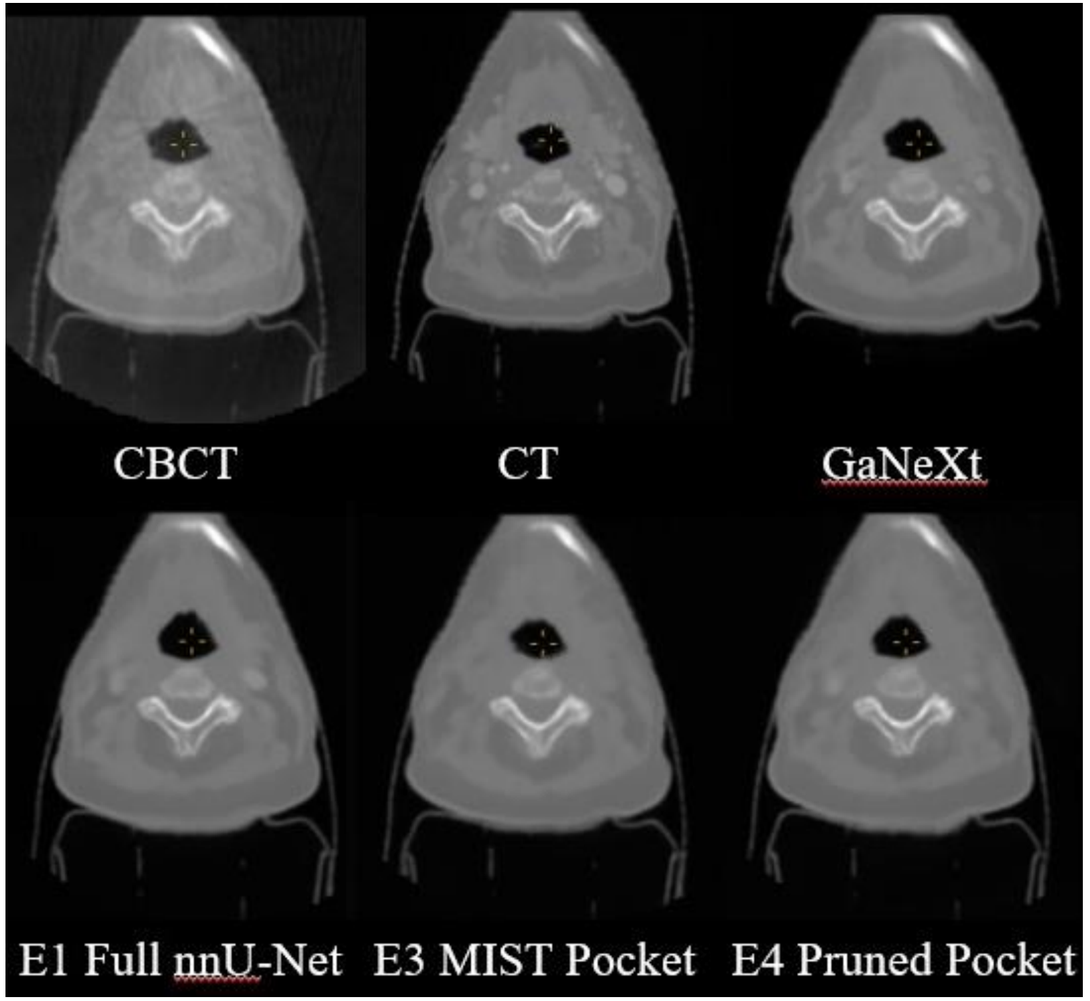
Model filter width pruning

- **Architectural change limiting filters per conv layer**

- **Instead of having nnunet standard filter density of for translation of 32-64-128-256-512-512 I did 16-24-32-64-128-128**

Table 5: Model size, inference speed and speed up factors for all models

Model		Inference speed (s)		Speed up factor	
	Model size (MB)	GPU	CPU	GPU	CPU
GaNeXt	224.2	72	5160	-	-
E1 Full nnU-Net	979.7	7	288	10.3	17.9
E2 Half-res	978.7	3	66	24.0	78.2
E3 MIST Pocket	13.7	3	112	24.0	46.1
E4 Pruned Pocket	**126.9**	**2**	**48**	**36**	**107.5**

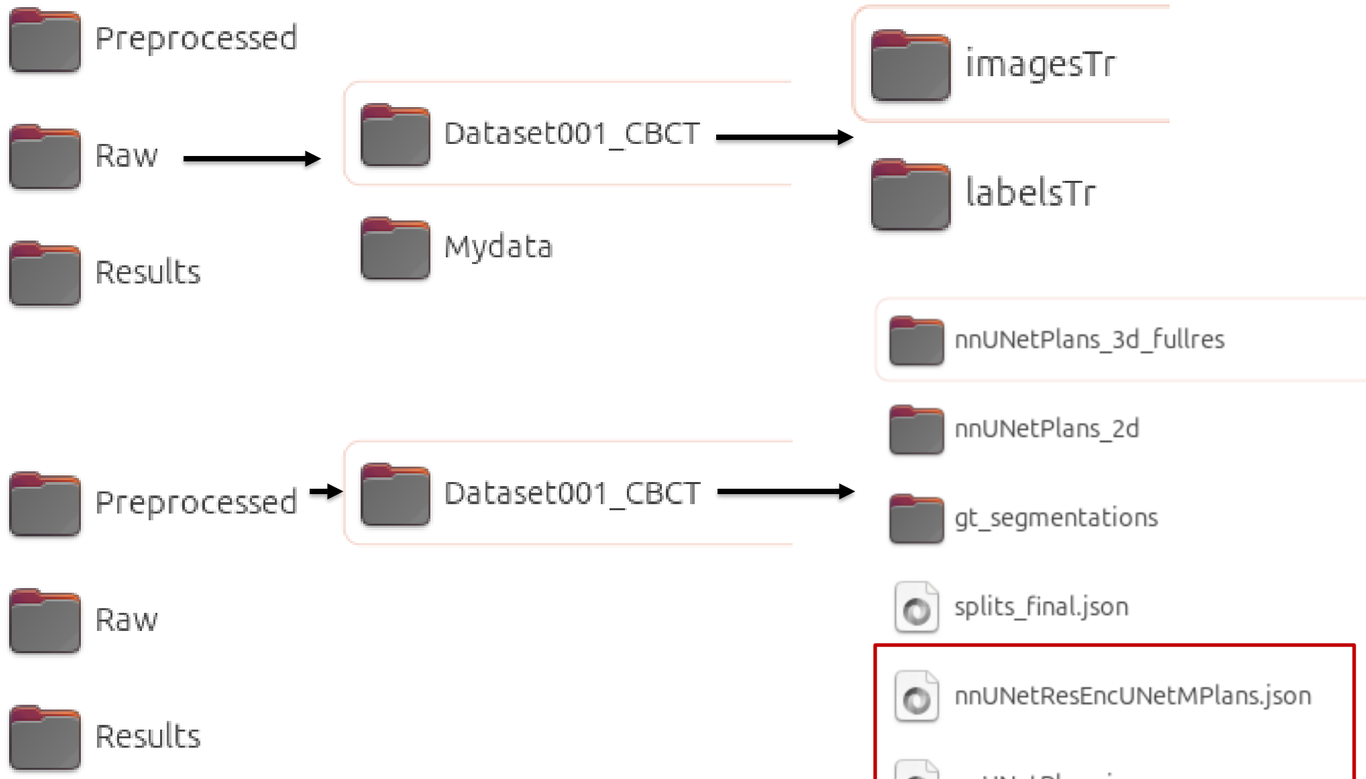


MAKING NNUNET LIGHTWEIGHT

- Nnunet already has AMP – making sure F32-bit is only used when necessary.
- V1 included a lowres configuration which has been removed in v2(you can down sample before preprocessing still.
- Nnunet configures the largest possible patch size for your data during preprocessing and experiment planning to provide the best global understanding of images.
- Nnunet has a modular design allowing us to alter things like:
  - Patch size
  - Architecture design

  before training

- HOW do I do it???? -->



- The plans.json file is created during experiment planning and contains the architecture and the patch size for 2d and 3d configs.

MAKING NNUNET LIGHTWEIGHT


```
{
    "dataset_name": "Dataset001_CBCT",
    "plans_name": "nnUNetResEncUNetMPlans",
    "original_median_spacing_after_transp": [
        3.0,
        1.0,
        1.0
    ],
    "original_median_shape_after_transp": [
        87,
        310,
        310
    ],
    "image_reader_writer": "SimpleITKIO",
    "transpose_forward": [
        0,
        1,
        2
    ],
```

```
"configurations": {
    "2d": {
        "data_identifier": "nnUNetPlans_2d",
        "preprocessor_name": "DefaultPreprocessor",
        "batch_size": 32,
        "patch_size": [
            320,
            320
        ],
        "median_image_size_in_voxels": [
            309.5,
            309.5
        ],
        "spacing": [
            1.0,
            1.0
        ],
        "normalization_schemes": [
            "CTNormalization"
        ],
        "use_mask_for_norm": [
            false
        ],
```

```
"architecture": {
    "network_class_name": "dynamic_network_architectures.architectures.unet.ResidualEncoderUNet",
    "arch_kwargs": {
        "n_stages": 7,
        "features_per_stage": [
            32,
            64,
            128,
            256,
            512,
            512,
            512
        ],
        "conv_op": "torch.nn.modules.conv.Conv2d",
        "kernel_sizes": [
            [
                3,
                3
            ],
            [
                3,
                3
            ],
            [
                3,
```

MAKING NNUNET LIGHTWEIGHT


```
{
    "dataset_name": "Dataset001_CBCT",
    "plans_name": "nnUNetResEncUNetMPlans",
    "original_median_spacing_after_transp": [
        3.0,
        1.0,
        1.0
    ],
    "original_median_shape_after_transp": [
        87,
        310,
        310
    ],
    "image_reader_writer": "SimpleITKIO",
    "transpose_forward": [
        0,
        1,
        2
    ],
```

```
"3d_fullres": {
    "data_identifier": "nnUNetPlans_3d_fullres",
    "preprocessor_name": "DefaultPreprocessor",
    "batch_size": 2,
    "patch_size": [
        48,
        192,
        192
    ],
    "median_image_size_in_voxels": [
        87.0,
        309.5,
        309.5
    ],
    "spacing": [
        3.0,
        1.0,
        1.0
    ],
    "normalization_schemes": [
        "CTNormalization"
    ],
    "use_mask_for_norm": [
        false
    ],
```

```
"architecture": {
    "network_class_name": "dynamic_network_architectures.architectures.unet.ResidualEncoderUNet",
    "arch_kwargs": {
        "n_stages": 6,
        "features_per_stage": [
            16,
            24,
            32,
            64,
            128,
            128
        ],
        "conv_op": "torch.nn.modules.conv.Conv3d",
        "kernel_sizes": [
            [
                1,
                3,
                3
            ],
            [
                3,
                3,
                3
            ],
```

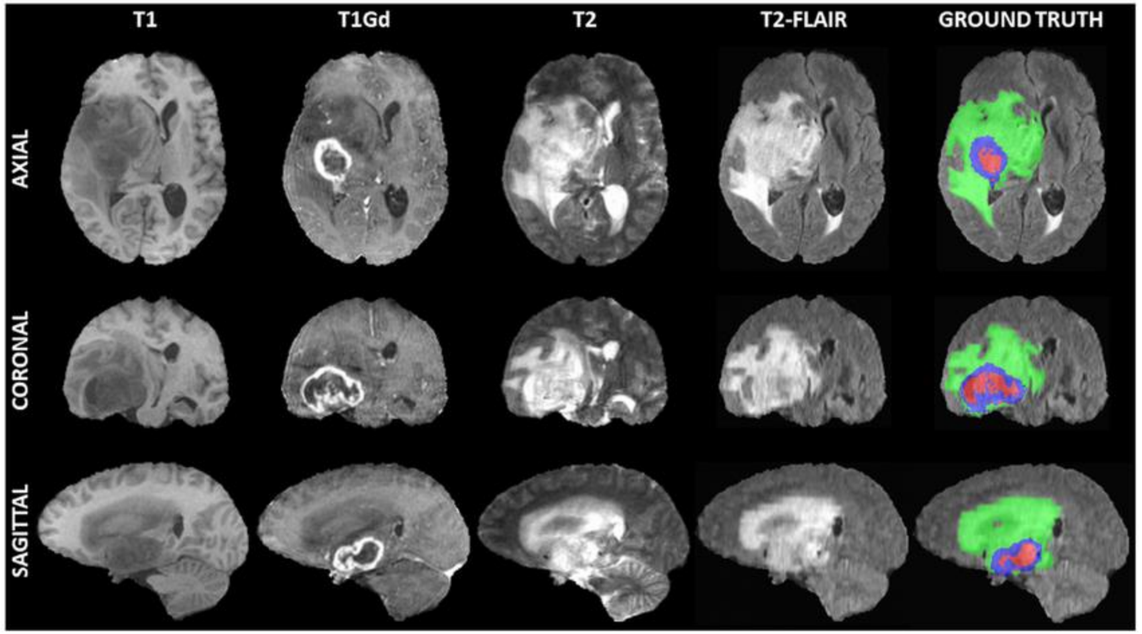
MAKING NNUNET LIGHTWEIGHT

```
############################################################################
Please cite the following paper when using nnU-Net:
Isensee, F., Jaeger, P. F., Kohl, S. A., Petersen, J., & Maier-Hein, K. H. (2021). nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation. Nature methods, 18(2),
203-211.
############################################################################

2026-03-05 14:19:20.633579: Using torch.compile...
2026-03-05 14:19:21.833027: do_dummy_2d_data_aug: True
2026-03-05 14:19:21.833688: Creating new 5-fold cross-validation split...
2026-03-05 14:19:21.835268: Desired fold for training: 0
2026-03-05 14:19:21.835381: This split has 41 training and 11 validation cases.

This is the configuration used by this training:
Configuration name: 3d_fullres
 {'data_identifier': 'nnUNetPlans_3d_fullres', 'preprocessor_name': 'DefaultPreprocessor', 'batch_size': 2, 'patch_size': [48, 192, 192], 'median_image_size_in_voxels': [87.0, 309.5, 309.5],
'spacing': [3.0, 1.0, 1.0], 'normalization_schemes': ['CTNormalization'], 'use_mask_for_norm': [False], 'resampling_fn_data': 'resample_data_or_seg_to_shape', 'resampling_fn_seg':
'resample_data_or_seg_to_shape', 'resampling_fn_data_kwargs': {'is_seg': False, 'order': 3, 'order_z': 0, 'force_separate_z': None}, 'resampling_fn_seg_kwargs': {'is_seg': True, 'order': 1,
'order_z': 0, 'force_separate_z': None}, 'resampling_fn_probabilities': 'resample_data_or_seg_to_shape', 'resampling_fn_probabilities_kwargs': {'is_seg': False, 'order': 1, 'order_z': 0,
'force_separate_z': None}, 'architecture': {'network_class_name': 'dynamic_network_architectures.architectures.unet.ResidualEncoderUNet', 'arch_kwargs': {'n_stages': 6, 'features_per_stage': [16,
24, 32, 64, 128, 128], 'conv_op': 'torch.nn.modules.conv.Conv3d', 'kernel_sizes': [[1, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3], [3, 3, 3]], 'strides': [[1, 1, 1], [1, 2, 2], [2, 2, 2],
[2, 2, 2], [2, 2, 2], [1, 2, 2]], 'n_blocks_per_stage': [1, 3, 4, 6, 6, 6], 'n_conv_per_stage_decoder': [1, 1, 1, 1, 1], 'conv_bias': True, 'norm_op':
'torch.nn.modules.instancenorm.InstanceNorm3d', 'norm_op_kwargs': {'eps': 1e-05, 'affine': True}, 'dropout_op': None, 'dropout_op_kwargs': None, 'nonlin': 'torch.nn.LeakyReLU', 'nonlin_kwargs':
{'inplace': True}}, '_kw_requires_import': ['conv_op', 'norm_op', 'dropout_op', 'nonlin']}, 'batch_dice': False}

These are the global plan.json settings:
 {'dataset_name': 'Dataset001_CBCT', 'plans_name': 'nnUNetResEncUNetMPlans', 'original_median_spacing_after_transp': [3.0, 1.0, 1.0], 'original_median_shape_after_transp': [87, 310, 310],
'image_reader_writer': 'SimpleITKIO', 'transpose_forward': [0, 1, 2], 'transpose_backward': [0, 1, 2], 'experiment_planner_used': 'nnUNetPlannerResEncM', 'label_manager': 'LabelManager',
'foreground_intensity_properties_per_channel': {'0': {'max': 3612.0, 'mean': 42.21962356567383, 'median': 39.15625, 'min': -1035.0, 'percentile_00_5': -993.0, 'percentile_99_5': 712.5, 'std':
169.25933837890625}}}

2026-03-05 14:19:23.394591: Unable to plot network architecture: nnUNet_compile is enabled!
2026-03-05 14:19:23.415119:
2026-03-05 14:19:23.415668: Epoch 0
2026-03-05 14:19:23.416353: Current learning rate: 0.01
2026-03-05 14:20:29.808843: train_loss 0.1601
2026-03-05 14:20:29.813910: val_loss 0.0409
2026-03-05 14:20:29.814262: Pseudo dice [np.float32(0.0), np.float32(0.0), np.float32(0.0), np.float32(0.0), np.float32(0.0)]
2026-03-05 14:20:29.814462: Epoch time: 66.4 s
2026-03-05 14:20:29.814623: Yayy! New best EMA pseudo Dice: 0.0
2026-03-05 14:20:32.556618:
```

Custom Kaggle lightweight Unet

- DEMO on BraTS 2021 Dataset
- Dataset has 1251 patients with each:
  - 4x 3D MRI images
  - 1xsegmentation map containing 3 segments

- Notebooks
  - Pre-processing
    - Extract data
    - Normalise
    - Crop-pad
    - Verification plot



Run this and save output as new private dataset to use for training.

Custom Kaggle lightweight Unet

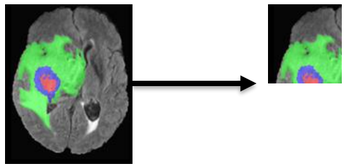
```python
CFG = {
    "data_dir": "/kaggle/input/datasets/williamwhy1/brats2021preprocessed/preprocessed",
    "fold": 0,
    "patch_size": (64, 64, 64),
    "features": [16, 32, 64, 128, 256],
    "foreground_prob": 0.01,          # % of patches will be forced to have tumor
    "foreground_threshold": 0.10,    # At least ?% of pixels must be tumor
    "region_based": False, # Toggle this to switch modes
    "num_classes": 3 if True else 4, # WT, TC, ET if region-based
    "batch_size": 1,
    "num_workers": 1,
    "device": torch.device("cuda" if torch.cuda.is_available() else "cpu"),

    "lr": 1e-5,
    "epochs": 10,
    "iters_per_epoch": 10,
    "ema_alpha": 0.90,
    "poly_power": 0.1,

    # Early Stopping & Subset Validation
    "val_every": 1,                 # Only validate every 5 epochs
    "patience": 3,                  # Wait 4 validations before killing run
    "min_delta": 0.01,              # 100% improvement required
    "val_percent": 0.03              # Validate on 100% of val set each epoch
}

print(f"Using device: {CFG['device']}")
```

- DEMO on BraTS 2021 Dataset
- Dataset has 1251 patients with each:
  - 4x 3D MRI images
  - 1xsegmentation map containing 3 segments

- Notebooks
  - Training
    - 3D with AMP training
    - Patches
    - Features

Custom Kaggle lightweight UNet

```python
CFG = {
    "data_dir": "/kaggle/input/datasets/williamwhy1/brats2021preprocessed/preprocessed",
    "fold": 0,
    "patch_size": (64, 64, 64),
    "features": [16, 32, 64, 128, 256],
    "foreground_prob": 0.01,          # % of patches will be forced to have tumor
    "foreground_threshold": 0.10,    # At least ?% of pixels must be tumor
    "region_based": False, # Toggle this to switch modes
    "num_classes": 3 if True else 4, # WT, TC, ET if region-based
    "batch_size": 1,
    "num_workers": 1,
    "device": torch.device("cuda" if torch.cuda.is_available() else "cpu"),

    "lr": 1e-5,
    "epochs": 10,
    "iters_per_epoch": 10,
    "ema_alpha": 0.90,
    "poly_power": 0.1,

    # Early Stopping & Subset Validation
    "val_every": 1,                 # Only validate every 5 epochs
    "patience": 3,                  # Wait 4 validations before killing run
    "min_delta": 0.01,              # 100% improvement required
    "val_percent": 0.03              # Validate on 100% of val set each epoch
}

print(f"Using device: {CFG['device']}")
```

- Foreground  vs 
- Region based training
  - Research shows might give better results for nested structures
- Limit number of iterations per epoch
  - Randomly sample a sub-population of 1000 training cases to train on to save time
- Exponential moving average for early stopping

- "SMART" validation for early stopping

Custom Kaggle lightweight Unet

- Print updates
  - 17.6s 10 Using device: cuda
  - 105.6s 11 🏃 Epoch 0 complete (Training only) loss: 1.4486
  - 186.4s 12 🏃 Epoch 1 complete (Training only) loss: 1.0527
  - 261.3s 13 🏃 Epoch 2 complete (Training only) loss: 0.8879
  - 333.7s 14 🏃 Epoch 3 complete (Training only) loss: 0.8174
  - 480.2s 15 📊 [VAL CHECK] EMA Dice -> WT: 0.6097 | TC: 0.3913 | ET: 0.4255 | Mean: 0.4755
  - 480.2s 16 ⭐ New Best Mean Dice! Saved EMA Model.

- Progress bars